

AI ASSISTENCE CODING- 7.5

SAI SPURTHI BELLAMKONDA

HT: 2303A51886

BT: 30

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Lab Objectives:

- To identify and correct syntax, logic, and runtime errors in Python programs using AI tools.
- To understand common programming bugs and AI-assisted debugging suggestions.
- To evaluate how AI explains, detects, and fixes different types of coding errors.
- To build confidence in using AI to perform structured debugging practices.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use AI tools to detect and correct syntax, logic, and runtime errors.
- Interpret AI-suggested bug fixes and explanations.
- Apply systematic debugging strategies supported by AI-generated insights.

Refactor buggy code using responsible and reliable programming patterns.

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.

The screenshot shows a VS Code editor with a file named 'Lab-7.5 ASS.py'. The code defines a function 'add_item' with a mutable default argument 'items=None'. Inside the function, 'items' is initialized as an empty list if it is None, then the item is appended, and the list is returned. The function is called twice with arguments 1 and 2. The terminal output shows the execution of the script, resulting in the output '[1]' and '[2]'. The status bar at the bottom indicates the file is at line 10, column 1, using UTF-8 encoding, and is a Python 3.8 (64-bit) file.

```
1 # Bug: Mutable default argument
2 def add_item(item, items=None):
3     if items is None:
4         items = []
5         items.append(item)
6     return items
7
8 print(add_item(1))
9 print(add_item(2))
10
```

```
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"
[1]
[2]
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

Bug: Floating point precision issue

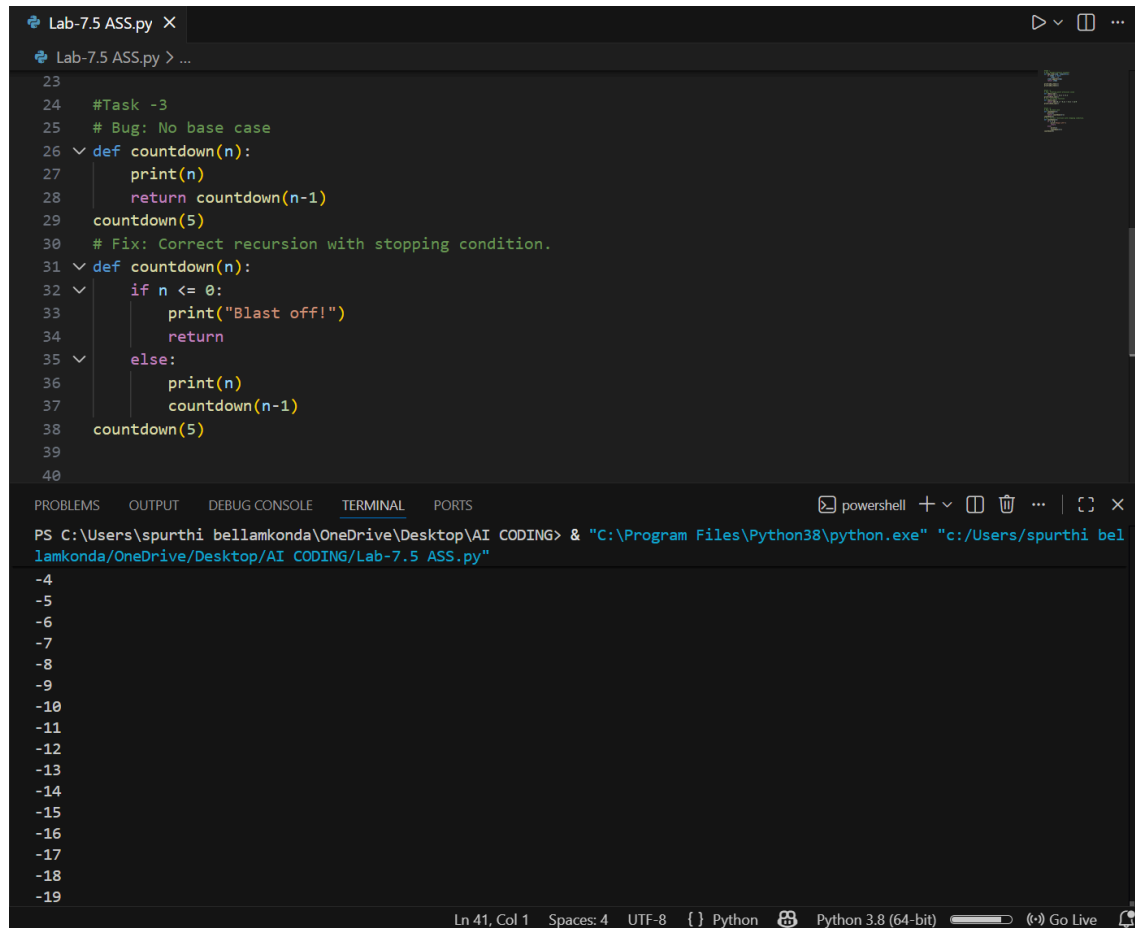
```
def check_sum():
    return (0.1 + 0.2) == 0.3
print(check_sum())
```

Expected Output: Corrected function

countdown(5)

Expected Output : Correct recursion with stopping condition

.



```
23
24 #Task -3
25 # Bug: No base case
26 def countdown(n):
27     print(n)
28     return countdown(n-1)
29     countdown(5)
30 # Fix: Correct recursion with stopping condition.
31 def countdown(n):
32     if n <= 0:
33         print("Blast off!")
34         return
35     else:
36         print(n)
37         countdown(n-1)
38     countdown(5)
39
40
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"

-4
-5
-6
-7
-8
-9
-10
-11
-12
-13
-14
-15
-16
-17
-18
-19

Ln 41, Col 1 Spaces: 4 UTF-8 { } Python Python 3.8 (64-bit) Go Live

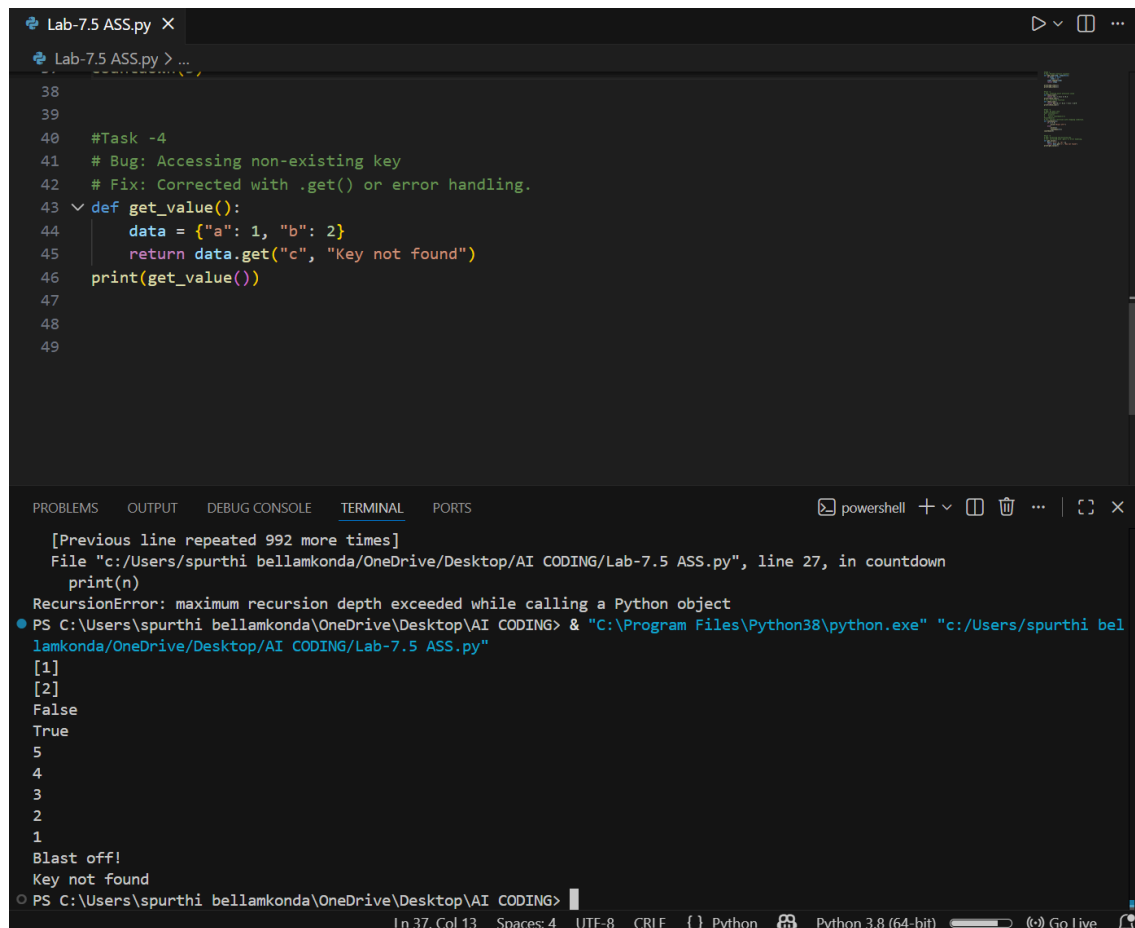
Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():
data = {"a": 1, "b": 2}
return data["c"]
print(get_value())
```

Expected Output: Corrected with .get() or error handling.



```
Lab-7.5 ASS.py X
Lab-7.5 ASS.py > ...
38
39
40 #Task -4
41 # Bug: Accessing non-existing key
42 # Fix: Corrected with .get() or error handling.
43 def get_value():
44     data = {"a": 1, "b": 2}
45     return data.get("c", "Key not found")
46 print(get_value())
47
48
49
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Previous line repeated 992 more times]
File "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py", line 27, in countdown
print(n)
RecursionError: maximum recursion depth exceeded while calling a Python object

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"

```
[1]
[2]
False
True
5
4
3
2
1
Blast off!
Key not found
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

In 37, Col 13 Spaces: 4 UTF-8 CRLF Python Python 3.8 (64-bit) Go Live

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():
```

```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

Expected Output: Corrected loop increments i.

The screenshot shows a VS Code editor window with a file named 'Lab-7.5 ASS.py'. The code in the editor is as follows:

```
56 def loop_example():
57     i = 0
58     while i < 5:
59         print(i)
60         i += 1
61     loop_example()
62
63
64 #Task -6
65 # Bug: Wrong unpacking
66 #a, b = (1, 2, 3)
67 #Fix: Correct unpacking or using _ for extra values.
68 a, b, _ = (1, 2, 3)
```

The bottom panel of the editor shows the 'TERMINAL' output. The command executed is: `PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"`. The output is:

```
[2]
False
True
5
4
3
2
1
Blast off!
Key not found
0
1
2
3
4
```

The status bar at the bottom indicates the current line is 68, column 20, with 4 spaces. The file encoding is UTF-8, and the line endings are CRLF. The editor is using Python 3.8 (64-bit).

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

```
def func():
```

```
    x = 5
```

```
    y = 10
```

```
    return x+y
```

Expected Output : Consistent indentation applied

```
Lab-7.5 ASS.py X
Lab-7.5 ASS.py > ...
69
70
71 #Task -7
72 # Bug: Mixed indentation
73 def func():
74     x = 5
75     y = 10
76     return x+y
77 #Fix: Consistent indentation
78 def func():
79     x = 5
80     y = 10
81     return x + y
82 print(func())

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"
3
2
1
Blast off!
Key not found
0
1
2
3
4
15
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
Ln 77, Col 2 Spaces: 4 UTF-8 CRLF Python Python 3.8 (64-bit) Go Live
```

Task 8 (Import Error – Wrong Module Usage)

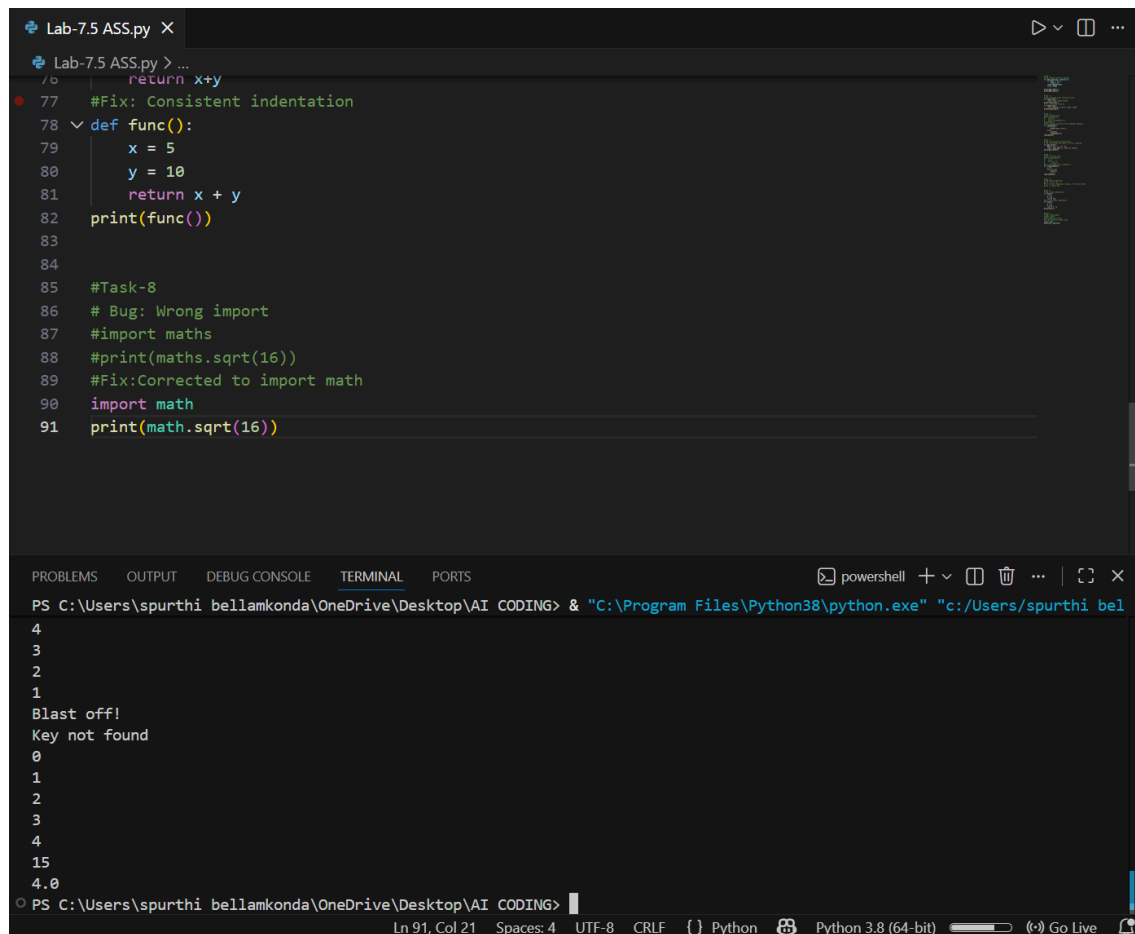
Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

import maths

print(maths.sqrt(16))

Expected Output: Corrected to import math



```
Lab-7.5 ASS.py > ...
76     return x+y
77 #Fix: Consistent indentation
78 def func():
79     x = 5
80     y = 10
81     return x + y
82 print(func())
83
84
85 #Task-8
86 # Bug: Wrong import
87 #import maths
88 #print(maths.sqrt(16))
89 #Fix:Corrected to import math
90 import math
91 print(math.sqrt(16))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bel
4
3
2
1
Blast off!
Key not found
0
1
2
3
4
15
4.0
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

```
def total(numbers):
```

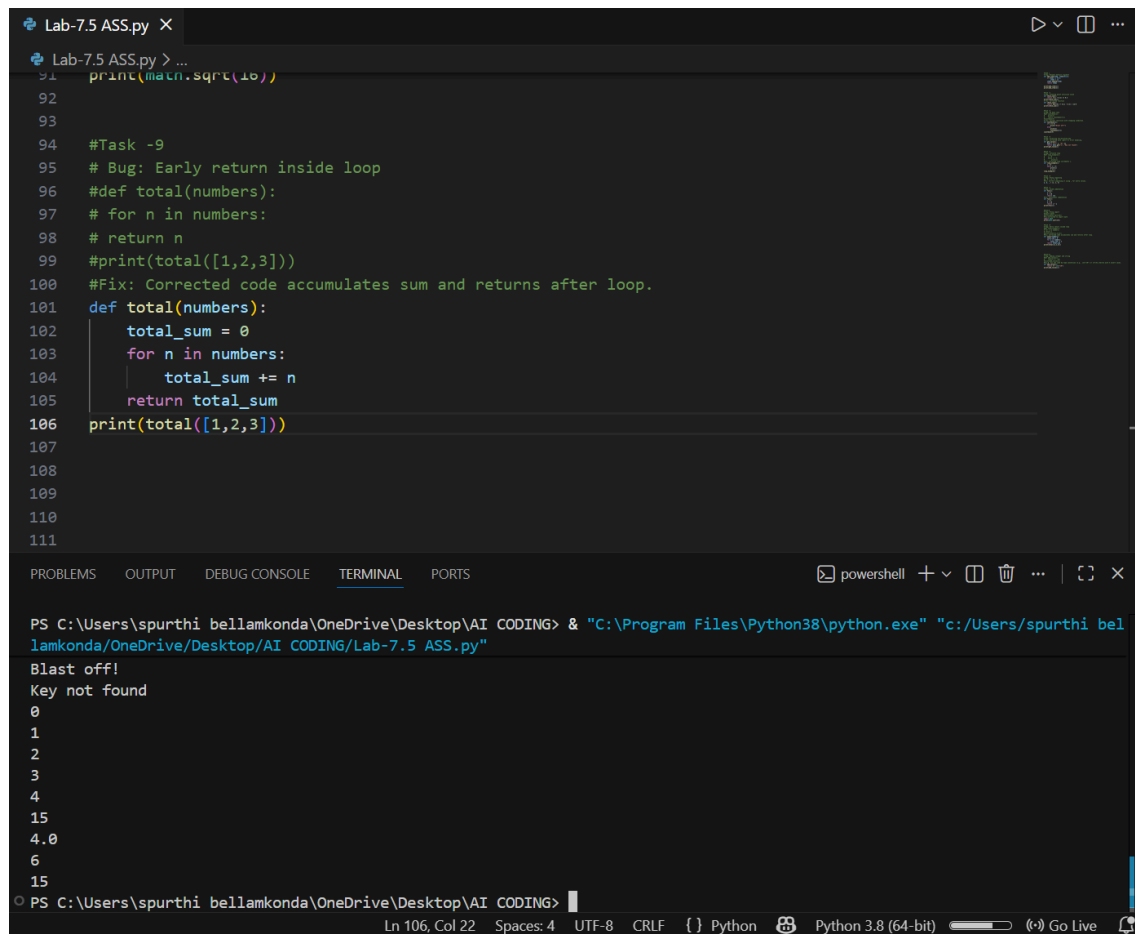
```
    for n in numbers:
```

```
        return n
```

```
    print(total([1,2,3]))
```

Expected Output:

Corrected code accumulates sum and returns after loop.



The screenshot shows a VS Code editor window with a file named 'Lab-7.5 ASS.py'. The code in the editor is as follows:

```
91 print(math.sqrt(16))
92
93
94 #Task -9
95 # Bug: Early return inside loop
96 #def total(numbers):
97 # for n in numbers:
98 # return n
99 #print(total([1,2,3]))
100 #Fix: Corrected code accumulates sum and returns after loop.
101 def total(numbers):
102     total_sum = 0
103     for n in numbers:
104         total_sum += n
105     return total_sum
106 print(total([1,2,3]))
107
108
109
110
111
```

The terminal at the bottom shows the command to run the script and its output:

```
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"
Blast off!
Key not found
0
1
2
3
4
15
4.0
6
15
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

The status bar at the bottom indicates the file is at Line 106, Column 22, using UTF-8 encoding with CRLF line endings, and is a Python 3.8 (64-bit) file.

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
return length * width
print(calculate_area())
```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.

- AI explanation of the bug.

Successful execution of assertions.

The screenshot shows a VS Code editor window with a file named 'Lab-7.5 ASS.py'. The code in the editor is as follows:

```

106 print(total([1,2,3]))
107
108
109 #Task -10
110 # Bug: Using undefined variable
111 #def calculate_area():
112 # return length * width
113 #print(calculate_area())
114 #Fix: * Corrected code with parameters.AI explanation of the bug.* Ask AI to identify th
115 def calculate_area(length, width):
116     return length * width
117 print(calculate_area(5, 10))
118 # Test cases
119 assert calculate_area(5, 10) == 50
120 assert calculate_area(3, 4) == 12
121 assert calculate_area(7, 2) == 14
122
123
124
125
126

```

Below the editor, the 'TERMINAL' panel shows the command prompt output:

```

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"
Key not found
0
1
2
3
4
15
4.0
6
50
15
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>

```

The status bar at the bottom indicates the file is at Line 114, Column 89, with 4 spaces, using UTF-8 encoding and CRLF line endings. The editor is configured for Python 3.8 (64-bit).

AI Explanation of the Bug:

The bug occurs because the function `calculate_area()` is using two variables (`length` and `width`) that are not defined

within the function. Python needs these variables to be passed to the function as arguments, but they are missing.

How the Fix Works:

To fix this:

1. We define `length` and `width` as parameters in the function.
2. Then, we pass values for these parameters when calling the function.
3. Finally, we added 3 `assert` test cases to make sure the function works correctly for different input values.

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values():
```

```
    return 5 + "10"
```

```
print(add_values())
```

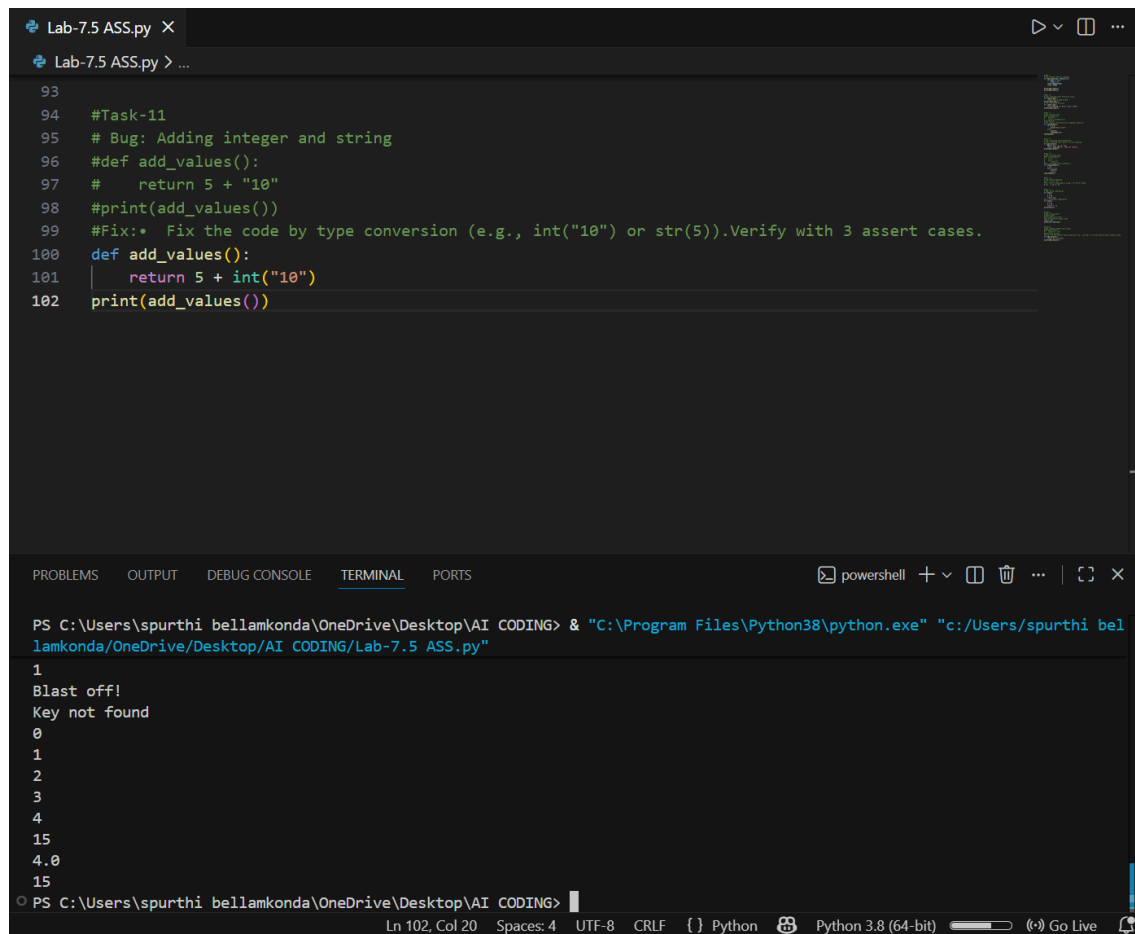
Requirements:

- Run the code to observe the error.
- AI should explain why int + str is invalid.
- Fix the code by type conversion (e.g., int("10") or str(5)).
- Verify with 3 assert cases.

Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.



The screenshot shows a VS Code editor window with a file named 'Lab-7.5 ASS.py'. The code in the editor is as follows:

```
93
94 #Task-11
95 # Bug: Adding integer and string
96 #def add_values():
97 #     return 5 + "10"
98 #print(add_values())
99 #Fix:• Fix the code by type conversion (e.g., int("10") or str(5)).Verify with 3 assert cases.
100 def add_values():
101     return 5 + int("10")
102 print(add_values())
```

Below the editor is a terminal window with the following output:

```
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"
1
Blast off!
Key not found
0
1
2
3
4
15
4.0
15
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

The status bar at the bottom indicates the file is at line 102, column 20, with 4 spaces, using UTF-8 encoding, CRLF line endings, and Python 3.8 (64-bit).

AI Explanation of the Bug:

The bug happens because Python doesn't allow adding an integer (5) and a string ("10") directly. These two types are incompatible for addition.

In Python, an integer is a number, and a string is a sequence of characters. When you try to add them together,

Python raises a `TypeError` because it doesn't know how to combine a number and a sequence of characters.

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

```
# Bug: Adding string and list
def combine():
    return "Numbers: " + [1, 2, 3]
print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```
Lab-7.5 ASS.py X
Lab-7.5 ASS.py > ...
128 #print(add_values())
129 #Fix:• Fix the code by type conversion (e.g., int("10") or str(5)).Verify with 3 assert cases.
130 def add_values():
131     return 5 + int("10")
132 print(add_values())
133
134
135 #Task- 12
136 # Bug: Adding string and list
137 #def combine():
138 # return "Numbers: " + [1, 2, 3]
139 #print(combine())
140 #Fix:• Fix using conversion (str([1,2,3]) or " ".join()).Verify with 3 assert cases.
141 def combine():
142     return "Numbers: " + str([1, 2, 3])
143 print(combine())
144
145
```

```
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"
0
1
2
3
4
15
4,0
6
50
15
Numbers: [1, 2, 3]
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

AI Explanation of the Bug:

The bug occurs because Python doesn't allow adding a string ("Numbers: ") and a list ([1, 2, 3]) directly. In Python, a string is a sequence of characters, and a list is a collection of items. Since these are two different data types, trying to add them together directly will raise a `TypeError`.

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

```
def repeat_text():  
    return "Hello" * 2.5  
print(repeat_text())
```

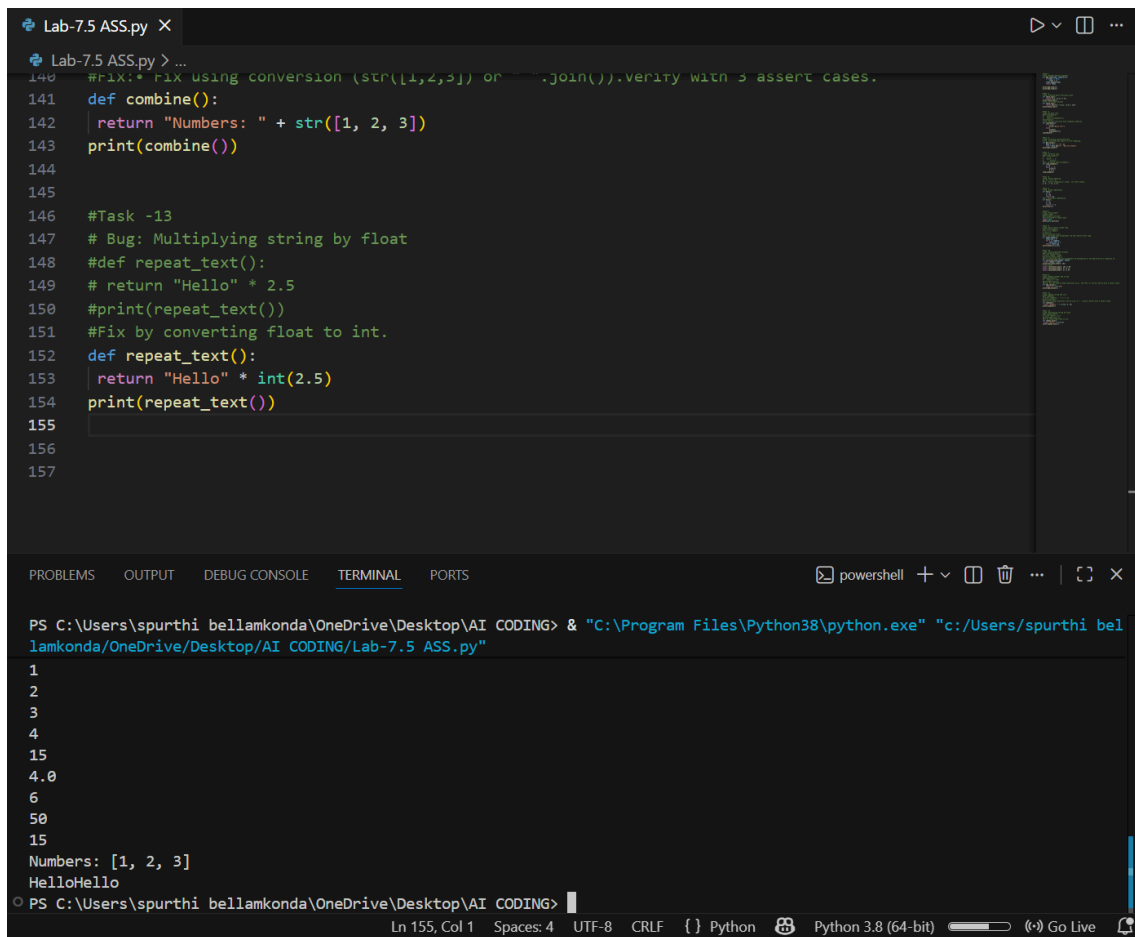
Requirements:

Observe the error.

Explain why float multiplication is invalid for strings.

Fix by converting float to int.

Add 3 assert test cases.



```
Lab-7.5 ASS.py X  
Lab-7.5 ASS.py > ...  
140 #Fix: * FIX using conversion (str([1,2,3]) or .join()). verify with 3 assert cases.  
141 def combine():  
142     return "Numbers: " + str([1, 2, 3])  
143     print(combine())  
144  
145  
146 #Task -13  
147 # Bug: Multiplying string by float  
148 #def repeat_text():  
149 # return "Hello" * 2.5  
150 #print(repeat_text())  
151 #Fix by converting float to int.  
152 def repeat_text():  
153     return "Hello" * int(2.5)  
154     print(repeat_text())  
155  
156  
157  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"  
1  
2  
3  
4  
15  
4.0  
6  
50  
15  
Numbers: [1, 2, 3]  
HelloHello  
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>  
Ln 155, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.8 (64-bit) Go Live
```

AI Explanation of the Bug:

The bug occurs because Python doesn't allow multiplying a string ("Hello") by a float (2.5). String multiplication only

works with an integer value, which determines how many times the string should be repeated. When you try to

multiply by a float, Python raises a `TypeError` , as it doesn't know how to handle fractional repetitions of a string.

How the Fix Works:

To fix this:

1. We convert the float 2.5 to an integer using `int(2.5)` .
2. This converts 2.5 into 2 , so the string "Hello" will be repeated 2 times.
3. Now, we can multiply the string by an integer without causing any errors.

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where `None` is added to an integer.

```
def compute():  
    value = None  
    return value + 10  
print(compute())
```

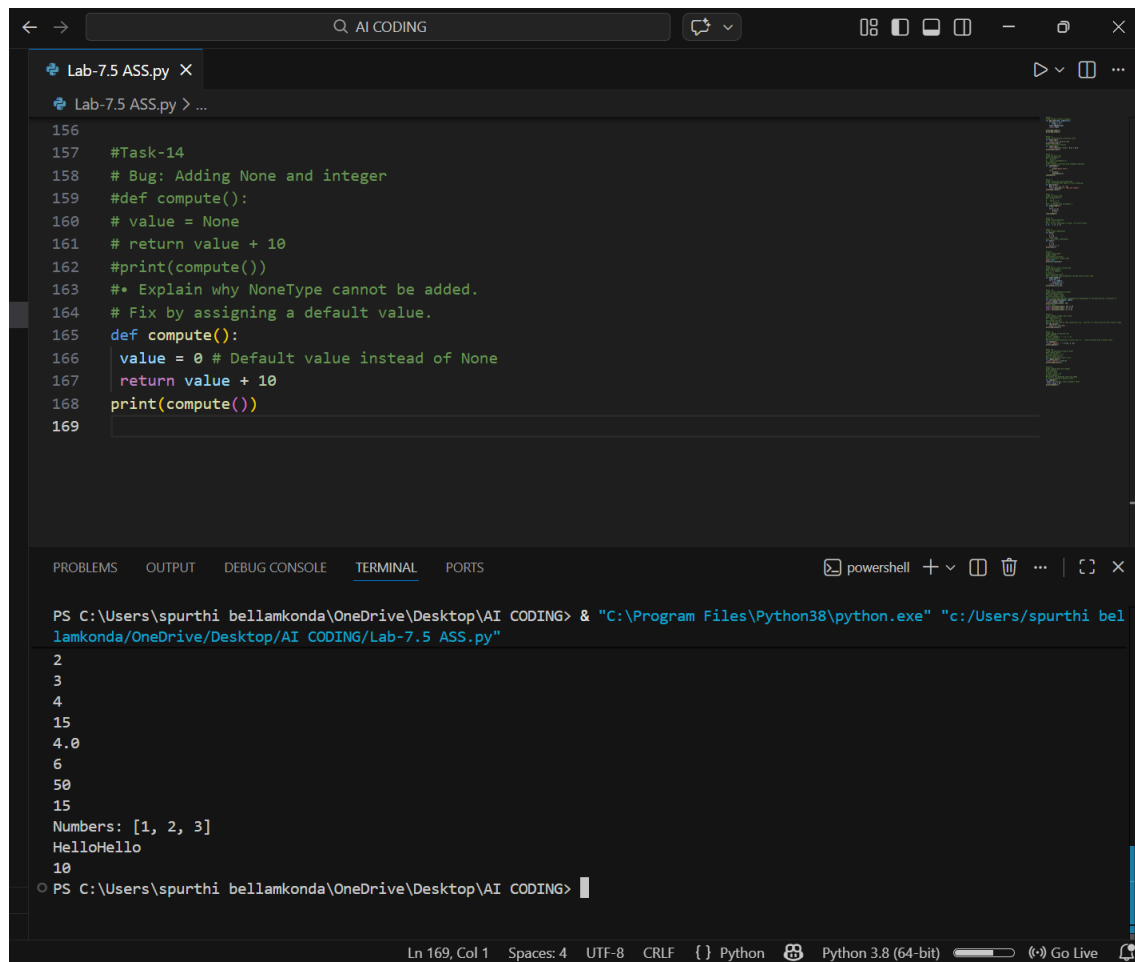
Requirements:

Run and identify the error.

Explain why `NoneType` cannot be added.

Fix by assigning a default value.

Validate using `asserts`.



The screenshot shows a VS Code editor window with a file named 'Lab-7.5 ASS.py'. The code in the editor is as follows:

```
156
157 #Task-14
158 # Bug: Adding None and integer
159 #def compute():
160 # value = None
161 # return value + 10
162 #print(compute())
163 #* Explain why NoneType cannot be added.
164 # Fix by assigning a default value.
165 def compute():
166     value = 0 # Default value instead of None
167     return value + 10
168 print(compute())
169
```

Below the editor is a terminal window showing the command to run the script and its output:

```
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"
2
3
4
15
4.0
6
50
15
Numbers: [1, 2, 3]
HelloHello
10
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

The status bar at the bottom indicates the current line and column (Ln 169, Col 1), encoding (UTF-8), line endings (CRLF), and the Python version (Python 3.8 (64-bit)).

AI Explanation of the Bug:

The bug happens because `None` is a special data type in Python, representing the absence of a value. When you try to add `None` to an integer (like `None + 10`), Python raises a `TypeError` because `None` cannot be directly combined with other data types like integers. The operation is undefined, as `None` is not considered a valid operand for arithmetic.

How the Fix Works:

To fix this:

1. We assign a default value, such as `0`, to `value` instead of leaving it as `None`.
2. This ensures that when we perform the addition (`value + 10`), both operands are valid (an integer plus another integer).

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

```
def sum_two_numbers():  
a = input("Enter first number: ")  
b = input("Enter second number: ")  
return a + b  
print(sum_two_numbers())
```

Requirements:

Fix using int() conversion.

Explain why input is always string.

Verify with assert test cases.

The screenshot shows a VS Code editor window with a file named 'Lab-7.5 ASS.py'. The code in the editor is as follows:

```
170  
171 #Task-15  
172 # Bug: Input remains string  
173 #def sum_two_numbers():  
174 # a = input("Enter first number: ")  
175 # b = input("Enter second number: ")  
176 # return a + b  
177 #print(sum_two_numbers())  
178 #Fix using int() conversion.  
179 def sum_two_numbers():  
180     a = int(input("Enter first number: "))  
181     b = int(input("Enter second number: "))  
182     return a + b  
183 print(sum_two_numbers())  
184
```

Below the editor is a terminal window. The command executed is: `PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-7.5 ASS.py"`. The terminal output shows the program's execution, including prompts for input and the final result:

```
3  
4  
15  
4.0  
6  
50  
15  
Numbers: [1, 2, 3]  
HelloHello  
10  
Enter first number: 16  
Enter second number: 65  
81  
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

The status bar at the bottom indicates the current line and column (Ln 183, Col 25), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), the language (Python), and the Python version (Python 3.8 (64-bit)).

AI Explanation of the Bug:

The bug happens because the input() function in Python always returns a string, regardless of what the user types.

So, when you try to add two inputs ($a + b$), Python is adding two strings together, not numbers. This results in string

concatenation instead of numerical addition.

For example, if the user enters 3 and 5 , the code would treat them as strings ("3" and "5") and concatenate them into "35" rather than adding them as numbers.